

Enunciado práctica laboratorio

Ruben.Cantarero@uclm.es

Curso 2025-2026

REGISTRO DE CAMBIOS – VERSIÓN 1.1 (16/02/2026)

- Se modifica el carácter obligatorio de la interconexión mediante VPN WireGuard entre gateway local y gateway server.
- La VPN pasa a considerarse ampliación voluntaria, con una bonificación de hasta +0,5 puntos sobre la calificación final.
- En ausencia de VPN, se permitirá la comunicación directa entre gateway local y gateway server, manteniendo la separación de brokers y el modelo edge/cloud descrito en el documento.
- En caso de discrepancia entre este registro de cambios y el texto del enunciado, prevalece lo indicado en este registro.

CONTEXTUALIZACIÓN

En esta práctica se plantea el diseño e implementación de un sistema de Internet de las Cosas (IoT) orientado a la seguridad y al control de acceso a espacios controlados, un caso de uso representativo de numerosos escenarios reales en ámbitos como edificios inteligentes, instalaciones industriales o infraestructuras críticas. El sistema permitirá gestionar el acceso a una puerta mediante identificación RFID y detectar situaciones de posible intrusión a partir del análisis del movimiento de la puerta, activando mecanismos de alarma y notificación cuando corresponda.

El sistema se concibe con fines docentes y se presenta de forma simplificada, pero preserva los elementos fundamentales de una arquitectura IoT real: dispositivos finales (nodos IoT), un gateway con capacidades de *edge computing* y una infraestructura centralizada que simula un entorno de *cloud computing*. El objetivo es que el estudiantado trabaje sobre un diseño realista y extrapolable, evitando soluciones ad hoc o excesivamente acopladas a un único escenario.

Aunque en este enunciado se considera una única red local, la arquitectura propuesta sigue un patrón escalable y habitual en despliegues IoT reales, donde múltiples redes locales, cada una con su propio gateway, se integran de forma segura en una infraestructura central. De este modo, el sistema desarrollado puede entenderse como una instancia concreta de un modelo que podría reproducirse en múltiples emplazamientos, manteniendo una lógica común y una gestión centralizada.

El gateway local desempeña un papel clave en el sistema, actuando como elemento de computación en el borde. Es responsable de concentrar la información procedente de los nodos IoT, correlacionar eventos heterogéneos (lecturas de tarjetas RFID y movimiento de la puerta), consultar información persistente en la infraestructura cloud y tomar decisiones en tiempo real, reduciendo la latencia y evitando delegar toda la inteligencia en los dispositivos finales o en la nube.

VISIÓN GENERAL DEL SISTEMA

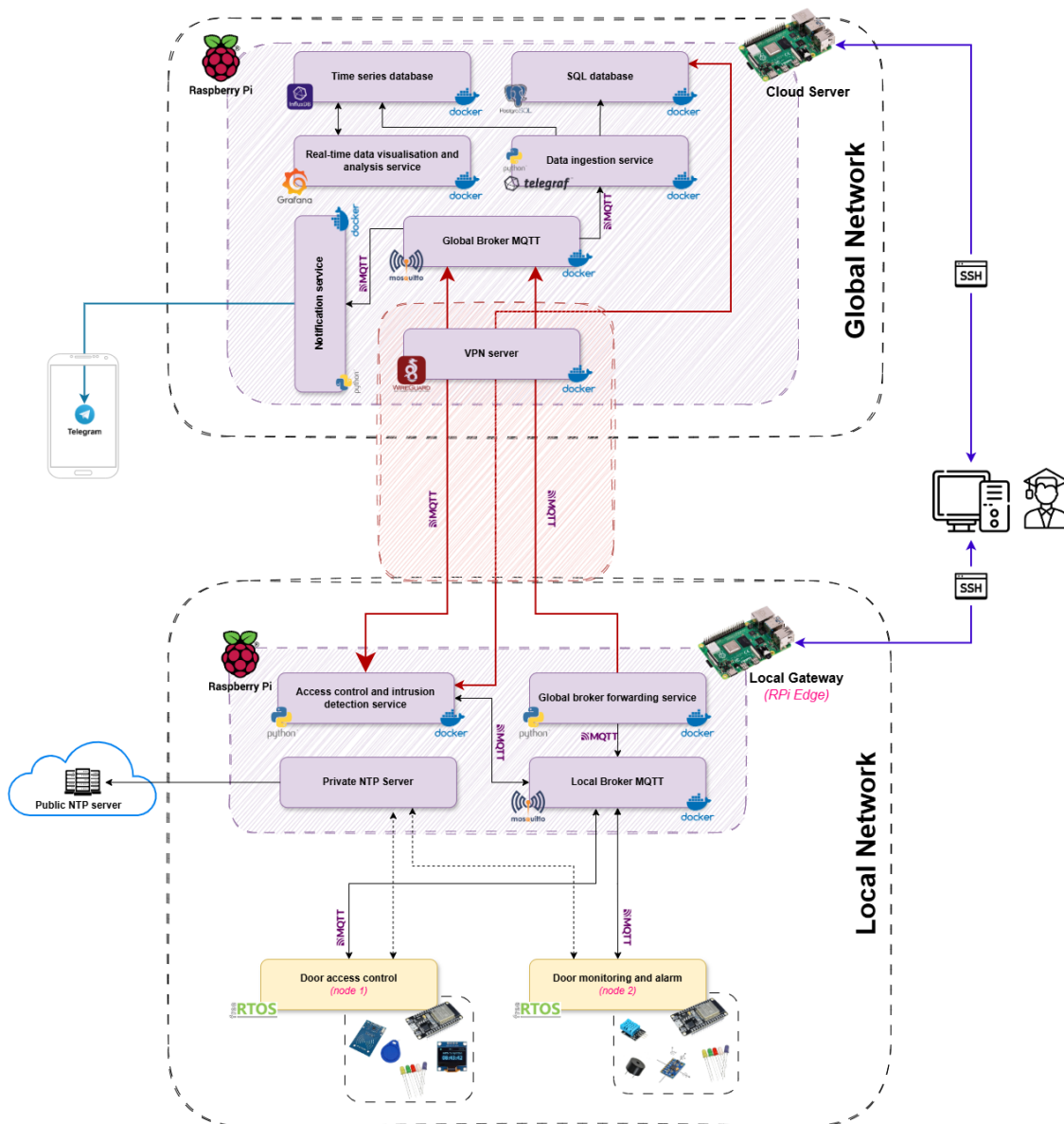
El sistema se organiza en tres capas: dispositivos finales (nodos IoT), gateway local (edge) e infraestructura central (cloud simulado):

- **Nodos IoT (ESP32):** dos dispositivos finales programados con FreeRTOS.
 - El **nodo 1** gestiona eventos de control de acceso mediante RFID.
 - El **nodo 2** monitoriza el estado de la puerta y el entorno mediante telemetría inercial (IMU) y telemetría de temperatura/humedad (DHT11), y además incluye actuadores (buzzer/LED) para señalización de alarma. Ambos nodos se comunican **exclusivamente** con el broker MQTT local del gateway mediante el modelo publicación/suscripción.
- **Gateway local (Raspberry Pi – edge computing):** punto de concentración de la red IoT local. Aloja el broker MQTT local, ofrece sincronización horaria (NTP) a los nodos y ejecuta, en contenedores Docker, la lógica de edge: suscripción a eventos, correlación de información (p. ej., accesos RFID vs movimiento de puerta), consulta de autorizaciones en la infraestructura central y emisión de comandos hacia los nodos cuando corresponde.
- **Cloud /gateway server (Raspberry Pi):** representa la infraestructura central. Aloja el broker MQTT global, la VPN (WireGuard), bases de datos (p. ej., PostgreSQL/InfluxDB) y servicios de visualización/monitorización.

Los nodos IoT no publican directamente en el broker global: toda la comunicación hacia la capa cloud pasa por el gateway local a través de la VPN. Este diseño reproduce un patrón habitual en despliegues reales, donde redes locales aisladas se integran de forma segura en una infraestructura central. Además, concentrar la lógica en el edge permite reducir latencia, limitar tráfico hacia cloud y mantener respuestas locales incluso ante fallos temporales de conectividad.

Aunque el escenario descrito utiliza una única red local, la arquitectura es escalable: en un despliegue real existirían múltiples redes (sites), cada una con su gateway local replicando el mismo esquema, mientras que la infraestructura cloud integraría todos los datos distinguiendo su origen.

A continuación se muestra un diagrama general del sistemas propuesto:



OBJETIVOS FORMATIVOS

La realización de esta práctica tiene como objetivo que el alumnado adquiera una visión integrada del diseño, desarrollo y despliegue de **sistemas IoT completos y realistas**, combinando programación en sistemas embebidos, comunicaciones, edge computing y servicios en la nube. En particular, se persigue que el estudiante sea capaz de:

- **Diseñar e implementar aplicaciones concurrentes en ESP32 utilizando FreeRTOS**, definiendo un conjunto coherente de tareas con responsabilidades diferenciadas (adquisición de sensores, agregación de datos, comunicación, control de actuadores), y empleando de forma adecuada mecanismos de sincronización y comunicación entre tareas.
- **Aplicar el protocolo MQTT en sistemas IoT**, utilizando correctamente el modelo de publicación/suscripción y diseñando una jerarquía de topics clara, legible y escalable, diferenciando el ámbito local (edge) del ámbito global (cloud).

- **Implementar lógica de edge computing en el gateway local**, capaz de correlacionar eventos heterogéneos (control de acceso, movimiento de la puerta, telemetría ambiental), consultar información persistente en la infraestructura cloud y tomar decisiones en tiempo casi real.
- **Diseñar y desplegar servicios distribuidos mediante contenedores Docker**, organizando adecuadamente los distintos componentes del sistema (broker MQTT, lógica de aplicación, bases de datos, visualización y notificaciones) y garantizando un despliegue reproducible mediante Docker Compose.
- **Gestionar correctamente el tiempo en sistemas distribuidos**, comprendiendo la importancia de la sincronización horaria, la deriva de los relojes en sistemas embebidos y el uso de NTP para generar marcas temporales coherentes que permitan la correlación de eventos entre dispositivos.
- **Integrar, validar y depurar el funcionamiento extremo a extremo del sistema IoT**, asegurando la consistencia de los datos, la robustez frente a errores y la correcta interacción entre nodos IoT, gateway local e infraestructura cloud.
- **Justificar decisiones de diseño técnico**, tales como la definición de umbrales de detección, ventanas temporales de correlación, estructura de mensajes y organización de servicios, utilizando criterios razonados y alineados con escenarios reales.

ARQUITECTURA DE COMUNICACIONES Y SEGURIDAD

La arquitectura de comunicaciones del sistema se basa en la separación de ámbitos y en el principio de mínima exposición, utilizando mecanismos habituales en despliegues IoT reales.

El sistema emplea dos brokers MQTT diferenciados:

1. **Broker MQTT local**, ejecutado en el gateway local, que constituye el único punto de comunicación accesible para los nodos IoT. Los dispositivos finales publican y reciben mensajes exclusivamente a través de este broker, evitando cualquier comunicación directa con la infraestructura cloud.
2. **Broker MQTT global**, ejecutado en el gateway server (cloud simulado), que centraliza la información procedente de uno o varios gateways locales y sirve como punto de integración con servicios de persistencia, visualización y notificación.

La interconexión entre el gateway local y el gateway server se realiza mediante una **VPN WireGuard**, gestionada mediante *wg-easy* en la infraestructura cloud. Este enlace reproduce un patrón ampliamente utilizado en entornos IoT reales: redes locales aisladas cuyos gateways establecen túneles seguros salientes hacia la infraestructura central, sin necesidad de exponer servicios locales a Internet.

Desde el punto de vista de la seguridad y del control del sistema:

- Los nodos IoT no conocen ni acceden al broker MQTT global.
- El gateway local actúa como frontera de seguridad, reenviando hacia cloud únicamente los eventos y la telemetría relevantes.

- Las decisiones de control (autorizaciones, activación de alarmas, comandos hacia los nodos) se toman en el gateway local, reduciendo la latencia y limitando la dependencia de la conectividad con cloud.

Este diseño permite escalar el sistema de forma natural a múltiples redes locales, manteniendo un esquema homogéneo de comunicaciones, reforzando el aislamiento entre redes y centralizando la gestión y supervisión en la infraestructura cloud.

GESTIÓN DEL TIEMPO Y SINCRONIZACIÓN HORARIA

La gestión correcta del tiempo es un aspecto crítico del sistema, ya que el gateway local debe correlacionar eventos procedentes de distintos nodos IoT, tales como intentos de acceso mediante RFID y movimiento de la puerta detectado por el sensor inercial. Para que esta correlación sea fiable, es imprescindible que todos los dispositivos compartan una referencia temporal coherente.

Con este objetivo, el sistema debe cumplir los siguientes requisitos:

- El gateway local debe sincronizar su reloj, al arrancar, con un servidor NTP público y actuar posteriormente como servidor NTP para la red IoT local durante toda la ejecución del sistema.
- Los nodos IoT deben sincronizar su reloj interno exclusivamente con el servidor NTP local, nunca con servidores públicos, reforzando así el aislamiento de la red y la coherencia temporal del sistema.
- Al arrancar, cada nodo IoT debe realizar una sincronización inicial de su reloj y generar marcas temporales en formato Unix (segundos). Además, cada nodo deberá resincronizar su reloj de forma periódica (por defecto, cada minuto¹) con el servidor NTP local, con el objetivo de corregir la deriva temporal propia de los microcontroladores, causada por la inestabilidad de sus osciladores y factores ambientales.
- Todos los mensajes publicados por los nodos IoT deben incluir un timestamp, generado en origen, que represente el instante de captura del evento o de la primera muestra del lote correspondiente.
- Los timestamps generados por los nodos no deben ser modificados por el gateway local al reenviar los mensajes hacia la infraestructura cloud, de forma que se preserve la trazabilidad temporal extremo a extremo.

Este enfoque permite analizar de manera consistente la secuencia de eventos del sistema, facilita la detección de situaciones anómalas y reproduce prácticas habituales en sistemas IoT reales, donde la coherencia temporal es un requisito esencial para la supervisión, auditoría y toma de decisiones automatizadas.

¹ Este intervalo de resincronización de un minuto se fija con fines académicos, para facilitar la corrección durante la práctica, mientras que en sistemas IoT reales dicho intervalo suele ser mayor y se ajusta en función de los requisitos de precisión y consumo energético.

IDENTIFICACIÓN DE NODOS

Con el objetivo de garantizar la trazabilidad, la correcta correlación de eventos y la escalabilidad del sistema, todos los nodos IoT deben estar identificados de forma única en los mensajes intercambiados mediante MQTT.

A tal efecto, en todos los mensajes JSON publicados por los nodos IoT, el campo `device_id` deberá contener la dirección MAC del ESP32, expresada como cadena de texto, y actuará como identificador único del dispositivo dentro del sistema.

Este identificador:

- permite distinguir de forma inequívoca el origen de cada evento o muestra de telemetría,
- facilita la integración de múltiples nodos y redes locales en la infraestructura cloud,
- y resulta esencial para tareas de depuración, auditoría y visualización en dashboards.

El valor de `device_id` deberá mantenerse constante durante toda la ejecución del sistema.

Ejemplo:

```
"device_id": "A4:CF:12:9B:3E:7F"
```

NODOS IoT (ESP32) — REQUISITOS GENERALES

Los dos nodos IoT del sistema deben implementarse sobre ESP32, utilizando un paradigma de ejecución concurrente basado en FreeRTOS. No se impone una arquitectura cerrada ni un número fijo de tareas; sin embargo, se evaluará positivamente que el diseño esté razonado y bien estructurado, evitando soluciones monolíticas o fuertemente acopladas.

En particular, se espera que el estudiante:

- desacople la adquisición de sensores de la comunicación MQTT,
- evite bloqueos innecesarios y retardos globales,
- y utilice de forma adecuada los mecanismos de FreeRTOS (tareas, colas, semáforos, notificaciones, etc.) cuando tenga sentido.

El diseño deberá reflejar una separación clara de responsabilidades, por ejemplo (a modo orientativo): adquisición de sensores, agregación de datos, publicación de eventos, recepción de comandos y control de actuadores.

Además de los eventos y la telemetría específicos de cada nodo, ambos nodos IoT deberán publicar periódicamente² un mensaje de estado, con el objetivo de facilitar la monitorización y el diagnóstico del sistema desde la infraestructura cloud.

Este mensaje de estado deberá incluir, al menos:

- el identificador del dispositivo (`device_id`),

² Para facilitar la corrección, este intervalo puede ser de un minuto.

- el tiempo de ejecución desde el último arranque (uptime_s, en segundos),
- y el número de reinicios del nodo (boot_count).

Formato JSON del mensaje de estado (referencia):

```
{
  "device_id": "A4:CF:12:9B:3E:7F",
  "ts": 1717581234,
  "boot_count": 7,
  "uptime_s": 5421
}
```

NODO 1 — CONTROL DE ACCESO A PUERTA

HARDWARE PROPORCIONADO

Este nodo se implementa sobre un ESP32 e integra los siguientes componentes:

- Lector RFID RC522.
- Pantalla OLED SSD1306.
- LEDs para señalización visual.

FUNCIONAMIENTO

El comportamiento del nodo reproduce el flujo típico de un sistema de control de acceso, actuando como dispositivo de captura, pero sin lógica de decisión local. En todo momento, la autorización del acceso depende del gateway local.

El funcionamiento esperado es el siguiente:

1. El nodo permanece en un estado de espera, mostrando en la pantalla un mensaje del estilo “Aproxime tarjeta”.
2. Al detectar una tarjeta RFID, el nodo:
 - muestra un estado transitorio, por ejemplo “Leyendo tarjeta...”,
 - obtiene el identificador de la tarjeta (UID),
 - genera un evento MQTT que incluye el UID y un timestamp generado a partir del reloj sincronizado por NTP,
 - y muestra un mensaje del tipo “Consultando permisos...”.
3. Tras la publicación del evento, el nodo espera una respuesta del gateway local, sin realizar ninguna validación local del acceso.
4. Al recibir la respuesta:
 - si el resultado indica autorización, el nodo muestra “Acceso autorizado” y activa la señalización visual correspondiente mediante LEDs,

- si el resultado indica denegación, muestra “Acceso no autorizado”,
- si se produce un error (timeout, respuesta mal formada o ausencia de respuesta), el nodo muestra un mensaje de error coherente (por ejemplo, “Error de comunicación”).

El diseño de los mensajes mostrados y el comportamiento exacto de los LEDs queda a criterio del estudiante, siempre que resulte coherente y consistente con el estado del sistema.

En ningún caso el nodo 1 debe decidir de forma autónoma si un acceso es válido o no; cualquier lógica de autorización implementada localmente se considerará incorrecta.

EJEMPLOS DE MENSAJES GENERADOS POR EL NODO

Evento RFID (publicado por el nodo):

```
{
  "device_id": "A4:CF:12:9B:3E:7F",
  "ts": 1717581201,
  "card_id": "A1B2C3D4"
}
```

Respuesta de autorización (recibida por el nodo desde el gateway):

```
{
  "ts": 1717581202,
  "card_id": "A1B2C3D4"
  "result": "AUTHORIZED"
}
```

NOTA. Valores típicos de result: "AUTHORIZED", "DENIED", "ERROR", etc.

NODO 2 — MONITORIZACIÓN DE PUERTA Y ALARMA

HARDWARE PROPORCIONADO

Este nodo se implementa sobre un ESP32 e integra los siguientes componentes:

- Sensor inercial MPU6050 (acelerómetro y giroscopio).
- Sensor de temperatura y humedad DHT11.
- Zumbador (*buzzer*) como actuador sonoro.
- LEDs como actuadores visuales (al menos un LED de alarma).

FUNCIONAMIENTO GENERAL Y TEMPORALIZACIÓN

El nodo 2 se encarga de la monitorización continua del estado de la puerta y del entorno, así como de la ejecución de acciones de alarma cuando recibe órdenes desde el gateway local. El nodo no toma decisiones de seguridad de forma autónoma.

El nodo publica dos tipos de telemetría, cada una como evento independiente y en formato batch, con el objetivo de reducir el número de publicaciones MQTT y aproximarse a patrones reales de agregación local:

- Telemetría inercial (IMU): el sensor se muestrea de forma periódica durante una ventana fija de 20 segundos, y todas las muestras recogidas durante ese intervalo se envían en un único evento batch.
- Telemetría de temperatura y humedad: las lecturas del sensor DHT11 se agregan igualmente durante una ventana de 20 segundos, pero se publican como un evento independiente del de la IMU.

Ambos eventos deben incluir:

- el identificador del dispositivo (`device_id`),
- el timestamp de la primera muestra del lote (`start_ts`),
- el periodo de muestreo (`sample_period_s`),
- y las muestras correspondientes al intervalo.

Además de publicar telemetría, el nodo 2 debe **suscribirse a un canal MQTT de comandos**, a través del cual el gateway local puede controlar la activación o desactivación de la alarma.

Los comandos esperados son:

- ALARM_ON
- ALARM_OFF

Cuando la alarma está activada, el nodo debe:

- activar el **LED de alarma** (encendido fijo o parpadeo, a criterio del estudiante),
- y hacer sonar el **buzzer** siguiendo el patrón temporal: **3 segundos activo / 1 segundo en silencio / 3 segundos activo / 1 segundo en silencio**, repetido indefinidamente.

Este comportamiento debe mantenerse **hasta recibir explícitamente** un comando ALARM_OFF. La alarma no debe desactivarse automáticamente ni por temporización local.

El diseño interno de las tareas FreeRTOS (adquisición de sensores, agregación de datos, recepción de comandos y control de actuadores) queda a criterio del estudiante, siempre que el comportamiento observable del nodo sea consistente con lo descrito.

MENSAJES GENERADOS POR EL NODO

Evento IMU (batch de 20 s, referencia):

```
{
  "device_id": "A4:CF:12:9B:3E:7F",
  "start_ts": 1717581200,
  "sample_period_s": 1,
  "samples": [
    { "ax": 0.1, "ay": 0.0, "az": 9.7, "gx": 0.02, "gy": 0.01, "gz": 0.00 },
    { "ax": 0.1, "ay": 0.0, "az": 9.7, "gx": 0.02, "gy": 0.01, "gz": 0.00 },
    ...
  ]
}
```

Evento de temperatura/humedad (batch de 20 s, referencia):

```
{
  "device_id": "A4:CF:12:9B:3E:7F",
  "start_ts": 1717581200,
  "sample_period_s": 1,
  "temperature_samples": [22.1, 22.2, 22.3, 22.2, ...],
  "humidity_samples": [60, 57, 58.1, 30.9, ...],
}
```

COMANDOS RECIBIDOS POR EL NODO

Comando de alarma (referencia):

```
{
  "ts": 1717581300,
  "cmd": "ALARM_ON"
}
```

Valores: "ALARM_ON" y "ALARM_OFF".

GATEWAY LOCAL (RPI EDGE) — SERVICIOS Y LÓGICA DE EDGE COMPUTING

El gateway local representa el componente de edge computing del sistema y constituye el punto central de control y decisión de la red IoT local. Su función no se limita al reenvío de mensajes, sino que debe concentrar la información procedente de los nodos IoT, correlacionar eventos, consultar información persistente en la infraestructura cloud y actuar en consecuencia.

La solución deberá organizarse como un conjunto de servicios desplegados mediante contenedores Docker, gestionados de forma reproducible mediante Docker Compose.

SERVICIOS MÍNIMOS EN EL GATEWAY LOCAL

Como mínimo, el gateway local deberá ejecutar los siguientes servicios:

- Broker MQTT local, ejecutado en un contenedor (por ejemplo, Mosquitto), que actúa como único punto de comunicación para los nodos IoT.
- Servidor NTP para la red local, sincronizado al arrancar contra un servidor NTP público y operativo durante toda la ejecución del sistema.
- Uno o varios servicios de aplicación, desplegados como contenedores Docker, que implementen:
 - la suscripción a los topics locales,
 - la lógica de edge (control de acceso e intrusión),
 - y el reenvío de eventos y telemetría hacia el broker MQTT global a través de la VPN.

La organización interna de estos servicios queda a criterio del estudiante, siempre que el despliegue sea automático, documentado y reproducible.

LÓGICA DE EDGE EXIGIDA

El gateway local debe implementar, al menos, la siguiente lógica:

a) Control de acceso (RFID)

Cuando recibe un evento RFID del nodo 1:

- consulta en el gateway server (cloud) si la tarjeta está autorizada (base de datos PostgreSQL),
- responde al nodo 1 con "AUTHORIZED" o "DENIED",
- y publica (o re-publica hacia cloud) un evento global de intento de acceso que permita auditoría y notificación.

b) Detección de intrusión correlacionando eventos

El gateway monitoriza la telemetría IMU del nodo 2 y debe detectar movimiento continuado de la puerta. Si detecta movimiento “sospechoso” (definición y umbrales a justificar por el estudiante) y no existe un evento de acceso autorizado cercano en el tiempo (instantes previos y posteriores, ventana temporal razonada), entonces:

- envía comando al nodo 2 para activar alarma,
- envía comando al nodo 1 para mostrar un mensaje del estilo “Se ha avisado a la policía” (simulado),
- y publica un evento en el broker MQTT global de alarma para que llegue a Telegram y quede persistido.

c) Reenvío a cloud

Toda la telemetría y eventos relevantes (IMU, temperatura/humedad, accesos, alarmas, estados) deben reenviarse al broker global manteniendo los timestamps de origen, tal y como se insistía en el enunciado anterior.

ARRANQUE DEL GATEWAY LOCAL

Al arrancar, el gateway local deberá:

1. sincronizar su reloj mediante NTP,
2. establecer la conexión VPN con la infraestructura cloud,
3. levantar el stack de contenedores necesarios,
4. y comenzar la suscripción a eventos locales y el reenvío hacia cloud.

GATEWAY SERVER (RPI CLOUD SIMULADO) — SERVICIOS OBLIGATORIOS

El gateway server representa la infraestructura central (cloud) del sistema IoT. Su función es integrar la información procedente de uno o varios gateways locales, proporcionar mecanismos de persistencia y visualización, y emitir notificaciones externas ante eventos relevantes. No asume lógica de control en tiempo real sobre los nodos IoT.

Todos los servicios del gateway server deberán desplegarse como contenedores Docker, gestionados de forma reproducible mediante Docker Compose.

Como mínimo, el gateway server deberá ejecutar los siguientes servicios:

- **Broker MQTT global** (por ejemplo, Mosquitto), que recibe los eventos y la telemetría reenviados desde los gateways locales y actúa como punto de integración con el resto de servicios cloud.
- **Servicio VPN WireGuard**, gestionado mediante *wg-easy*, que permite a los gateways locales establecer túneles seguros hacia la infraestructura cloud sin exponer servicios locales a Internet.
- **Base de datos InfluxDB**, destinada al almacenamiento de series temporales, tales como:
 - telemetría inercial (IMU),
 - lecturas de temperatura y humedad,
 - mensajes de estado de los nodos IoT.
- **Base de datos PostgreSQL**, destinada al almacenamiento de información relacional, incluyendo al menos:
 - autorizaciones de tarjetas RFID,
 - metainformación de la infraestructura (nodos, gateways, etc.), si procede.
- **Servicio de ingestión de datos**, encargado de consumir mensajes del broker MQTT global y persistirlos en InfluxDB o PostgreSQL según corresponda. Este servicio podrá implementarse mediante herramientas existentes (por ejemplo, Telegraf) o como un servicio propio desarrollado por el estudiante, siempre que su funcionamiento sea reproducible y esté correctamente documentado.
- **Grafana**, con uno o varios dashboards que permitan visualizar de forma clara la información recopilada por el sistema, incluyendo:

- telemetría histórica (IMU, temperatura/humedad),
 - últimos eventos de acceso,
 - últimos eventos de alarma,
 - y mensajes de estado de los nodos IoT (boot_count, uptime).
- **Servicio de notificaciones por Telegram**, implementado en Python y desplegado como contenedor Docker, que escuche eventos relevantes del broker MQTT global y envíe notificaciones:
 - por cada intento de acceso, indicando al menos el identificador de la tarjeta, la hora y si fue autorizado,
 - y por cada evento de alarma, incluyendo la hora y un contexto mínimo del suceso.

La correcta integración de estos servicios y su funcionamiento conjunto forma parte esencial de la evaluación de la práctica.

MQTT — DISEÑO DE CANALES, EJEMPLOS Y ESCALABILIDAD

El diseño de los canales MQTT (topics) constituye una parte fundamental de la práctica y será objeto de evaluación. Se espera que el alumnado proponga una estructura de topics clara, legible y escalable, alineada con patrones habituales en sistemas IoT reales.

En el sistema se distinguen dos ámbitos claramente diferenciados:

- **Broker MQTT local (edge)**: utilizado exclusivamente para la comunicación entre los nodos IoT y el gateway local. En este ámbito no es necesario identificar explícitamente la red local, ya que solo existe una.
- **Broker MQTT global (cloud)**: utilizado para la integración de información procedente de uno o varios gateways locales. En este caso, los topics deberán incluir un identificador de red o emplazamiento (site_id), con el objetivo de distinguir el origen de los datos en escenarios con múltiples redes.

Se espera que el diseño de topics:

- refleje claramente el tipo de información (estado, eventos, telemetría, comandos),
- evite ambigüedades semánticas,
- y facilite la suscripción selectiva por parte de los distintos servicios.

Ejemplos de estructura de topics

A modo orientativo, se muestran a continuación ejemplos de posibles estructuras de topics. Estas propuestas no son obligatorias, pero ilustran el tipo de organización esperada.

- **Broker local (edge)**:
 - edge/status/<mac>
 - edge/events/rfid/<mac>

- edge/telemetry/imu/<mac>
- edge/telemetry/temperatura_humidity/<mac>
- edge/cmd/access/<mac>
- edge/cmd/alarm/<mac>
- Broker global (cloud):
 - cloud/<site_id>/status/<mac>
 - cloud/<site_id>/telemetry/imu/<mac>
 - cloud/<site_id>/telemetry/temperature_humidity/<mac>
 - cloud/<site_id>/events/access
 - cloud/<site_id>/events/alarm

El alumnado podrá proponer variantes o extensiones de esta estructura, siempre que estén correctamente justificadas y mantengan la idea fundamental de que la infraestructura cloud debe poder identificar el origen de los datos por red local y tipo de información.

Diseños de topics planos, inconsistentes o que no distingan adecuadamente entre ámbitos, tipos de mensajes o direccionalidad (uplink/downlink) afectarán negativamente a la evaluación.

ORGANIZACIÓN DEL TRABAJO Y REPOSITORIOS

La práctica se realizará en grupos de tres personas. No se admitirán grupos de mayor tamaño. En caso de que un grupo esté formado por menos de tres estudiantes, se aplicará la siguiente penalización sobre la calificación final:

- **3 personas:** sin penalización.
- **2 personas:** penalización de **1,5 puntos**.
- **1 persona:** penalización de **2,5 puntos**.

El seguimiento y la evaluación del trabajo se realizarán exclusivamente mediante GitHub Classroom³. No se admitirán entregas por correo electrónico ni por ningún otro medio distinto al indicado.

Cada grupo dispondrá de tres repositorios independientes, asignados a través de GitHub Classroom, con el objetivo de separar claramente las responsabilidades de cada parte del sistema:

- **iot.nodes:** desarrollo del software de los nodos IoT basados en ESP32.
- **edge-gateway:** desarrollo y despliegue del gateway local (edge computing).
- **cloud-server:** desarrollo y despliegue de la infraestructura cloud simulada.

Todo el código fuente, scripts, ficheros de configuración, descriptores de contenedores y documentación necesaria para el despliegue y ejecución del sistema deberá encontrarse

³ El enlace de invitación de cada repositorio está disponible en Campus Virtual.

versionado en el repositorio correspondiente. No se valorará código o configuraciones externas al control de versiones.

La correcta organización del trabajo entre repositorios, así como la coherencia entre ellos, formará parte de la evaluación de la práctica.

DESPLIEGUE Y AUTOMATIZACIÓN

El despliegue del sistema en las Raspberry Pi deberá realizarse de forma **reproducible y automatizada**, sin depender de configuraciones manuales no documentadas ni de intervenciones ad hoc durante la ejecución.

Para ello, se utilizará **acceso remoto mediante SSH**, y será necesario configurar adecuadamente las [llaves de despliegue](#) que permitan a cada repositorio interactuar con la Raspberry Pi correspondiente. La correcta gestión de estas claves (creación, uso y documentación) forma parte del trabajo del estudiante y será tenida en cuenta en la evaluación.

Cada repositorio asociado a una Raspberry Pi (gateway local y gateway server) deberá incluir todo lo necesario para levantar los servicios correspondientes, de forma que el sistema pueda:

- desplegarse desde cero,
- reiniciarse tras una parada,
- y recuperarse de forma controlada ante fallos.

Este despliegue deberá realizarse, por ejemplo, mediante **Docker Compose**, incluyendo los descriptores de contenedores, ficheros de configuración y scripts necesarios. No se aceptarán soluciones que requieran pasos manuales no reflejados en el repositorio o que no puedan reproducirse a partir del contenido versionado.

La capacidad de automatizar correctamente el despliegue y de documentar de forma clara el proceso será considerada un indicador de la madurez técnica de la solución implementada.

ENTREGA, SEGUIMIENTO Y NORMAS DE EVALUACIÓN

No se tendrán en cuenta prácticas entregadas fuera de plazo (plazo indicado en campus Virtual). No se admitirán prácticas entregadas por correo electrónico ni por ningún otro medio distinto a GitHub Classroom.

El profesor realizará el seguimiento y la evaluación del trabajo desarrollado por el alumnado exclusivamente a través de los repositorios de GitHub Classroom asociados a la práctica. Para ello, el estudiante deberá seguir las instrucciones que se irán proporcionando a lo largo del curso en Campus Virtual.

No se evaluará ningún commit cuya fecha sea posterior a la fecha límite de entrega establecida. En caso de que algún estudiante no esté correctamente registrado en GitHub Classroom, deberá comunicarlo al profesor de prácticas con la suficiente antelación para poder resolver la incidencia.

NOTA IMPORTANTE SOBRE EL USO DE GIT

Se tendrán en cuenta las pautas de actividad de los repositorios, penalizando el uso indebido de los mismos, como por ejemplo la subida del código de forma masiva justo al finalizar la práctica. Se valorará positivamente un uso correcto, progresivo y continuado de los repositorios, que refleje el desarrollo real del trabajo a lo largo del tiempo.

CRITERIOS DE EVALUACIÓN

La evaluación de la práctica se realizará atendiendo a dos aspectos fundamentales: funcionalidad y calidad.

FUNCIONALIDAD

La funcionalidad valora que el sistema resuelva el problema conforme a las especificaciones descritas en este enunciado y que lo haga de forma consistente. Esto implica que el sistema debe funcionar adecuadamente en todas y cada una de las ejecuciones.

Si un comportamiento solo funciona de manera ocasional, se considerará que no funciona. La ausencia de mecanismos de gestión de errores, la existencia de casos no contemplados o comportamientos inconsistentes afectarán negativamente a la calificación, incluso aunque el sistema no llegue a fallar de forma explícita.

CALIDAD

La calidad del código se evaluará atendiendo, entre otros, a los siguientes aspectos:

- Estructura y organización del código mediante funciones, clases o módulos.
- Uso de nombres significativos y coherentes para variables, funciones y métodos, evitando código que no esté en inglés.
- Evitar el uso innecesario de variables globales.
- Evitar anidaciones injustificadas de estructuras de control.
- Evitar el uso incorrecto de tipos de datos, código redundante o código muerto.
- Evitar bloques de código excesivamente largos.

La valoración de la calidad es negativa, es decir, restará sobre la nota final obtenida en funcionalidad. Esto implica que una práctica completamente funcional puede no alcanzar la máxima calificación si la calidad del código no es adecuada.

Es importante destacar que el alumnado deberá ser capaz de defender el trabajo presentado. En cualquier momento se podrá solicitar la explicación o defensa de la solución implementada.

CONSIDERACIONES SOBRE EL USO DEL MATERIAL Y EL DESARROLLO FUERA DEL LABORATORIO

El material hardware necesario para la realización de esta práctica (placas ESP32, Raspberry Pi, sensores y actuadores asociados) **no podrá sacarse del laboratorio** bajo ningún concepto. Por tanto, todas aquellas partes del desarrollo que dependan directamente de dicho material deberán realizarse **presencialmente en el laboratorio**, aprovechando tanto el tiempo asignado

en clase como la asistencia y orientación proporcionada por el profesor durante las sesiones de trabajo.

Se recomienda encarecidamente que el alumnado planifique adecuadamente el trabajo en laboratorio, priorizando durante estas sesiones las tareas que requieren acceso al hardware físico, y resolviendo dudas técnicas directamente con el profesor.

No obstante, con el objetivo de facilitar el desarrollo progresivo del sistema y permitir la continuidad del trabajo fuera del aula, se contemplan las siguientes posibilidades:

- El estudiante podrá simular parcialmente los nodos IoT, implementando *dummies* o generadores de eventos que inyecten mensajes en los canales MQTT correspondientes. Esta simulación deberá respetar los formatos y estructuras definidos en el enunciado y permitirá avanzar en el desarrollo de la lógica del gateway y de la infraestructura cloud sin necesidad de disponer del hardware físico.
- Asimismo, el alumnado podrá reproducir en su propio ordenador personal los servicios desplegados en contenedores (broker MQTT, bases de datos, servicios de ingestión, etc.), con el fin de continuar el desarrollo y las pruebas fuera del laboratorio.

El uso de estas estrategias de simulación y despliegue local no es obligatorio. Un estudiante puede optar a la máxima calificación realizando el desarrollo íntegramente con el material del laboratorio y sin recurrir a simulaciones ni despliegues adicionales en su equipo personal.

No obstante, la correcta implementación y uso de mecanismos de simulación de nodos o de despliegues locales en el ordenador personal del estudiante se valorará positivamente como mérito adicional, al reflejar una mayor autonomía, comprensión del sistema y capacidad de reproducir entornos IoT complejos.